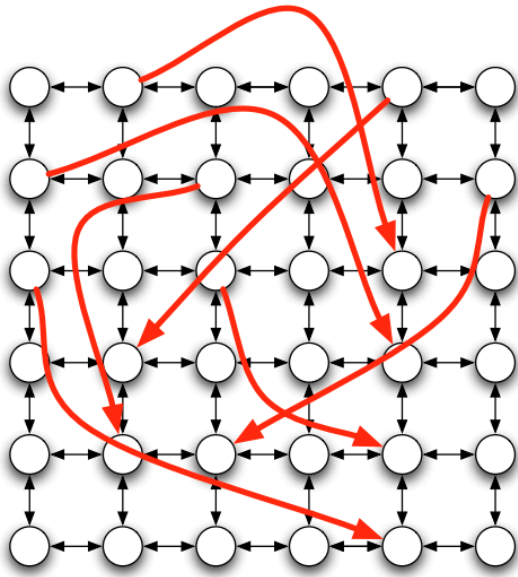
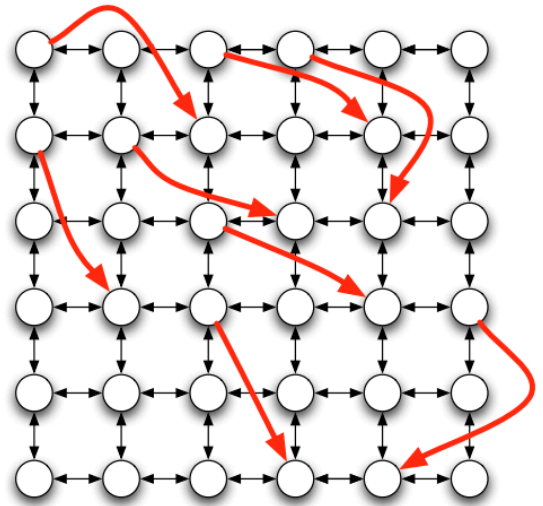


# L11: Decentralized Search Problem



(a) A small clustering exponent



(b) A large clustering exponent

Image Source: Textbook *"Networks, Crowds and Markets"* ↗ (<https://www.cs.cornell.edu/home/kleinber/networks-book>) by Easley and Kleinberg. Figure 20.5

Milgram's experiment has been repeated by others, both in offline and online social networks. These studies show two points:

- 1) Even global social networks (Facebook has around one billion users) have multiple short paths to reach almost anyone in the network (*typically around 5-6 hops, and almost always less than 10 hops*).
- 2) It is possible to find at least one of these short paths with strictly local information about the name and location of your neighbors, without having a global "map" (topology) of the network – and without a central directory that stores everyone's name and location in the network.

The first point should not be surprising to us by now. Recall what we learned about Small World networks in Lesson-5. A network can have both strong clustering (*i.e., many triangles, just as a regular network*) and short paths (increasing logarithmically with the size of the network, just as  $G(n,p)$  networks). The way this is achieved in the **Watts-Strogatz model** is that we start from a regular network with strong clustering and rewire a small fraction (say 1%) of edges to connect two random nodes. These random connections connect nodes that can be anywhere in the network – and so for large networks, these random connections provide us with **"long-range shortcuts"**. If you do not recall these points, please review Lesson-5.

Let us now focus on the second point: how is it possible to find a target node in a network with only local information, through a distributed search process? First, let us be clear about the information that each node has: a node  $v$  knows its own location in the network (*in some coordinate system*), the locations of only the nodes it is connected to, and the location of the target node.

The metric we will use to evaluate this distributed search process is the **"expected delivery time"**, i.e., the expected number of steps required to reach the target, over randomly chosen source and target nodes, and over a randomly generated set of long-range links.

To answer the previous question, we will consider an elegant model developed by Jon Kleinberg. Suppose that we start with a regular grid network in  $k$ -dimensions (*the visualizations shown refer to  $k=2$  but you can imagine mesh networks of higher dimensionality*). Each node is connected to its nearest neighboring nodes in the grid. Note that this network does not have cliques of size-3 (*triangles*) but it still has strong clustering in groups of nodes that are within a distance of two hops from each other.

Let  $d(v, w)$  be the distance between nodes  $v$  and  $w$  in this grid. Kleinberg's model adds a random edge out of  $v$ , pointing to a node  $w$  with a probability that is proportional to  $d(v, w)^{-q}$  – the exponent  $q$  is the key parameter of the model as we will see shortly.

The value of  $q$  controls how “spread out” these random connections actually are. If  $q=0$ , the random connections can connect any two nodes, independent of their distance (this is the case in the Watts-Strogatz model) – and so they are not very useful in decentralized search, when you try to “converge” to a certain target in as few steps as possible. If  $q$  is large, on the other hand, the random connections only connect nearby nodes – and so they will not provide the long-range shortcuts that are necessary for Small World property.

The visualization contrasts the case of a small  $q$  and a large  $q$ . What do you think would be the optimal value of  $q$ , so that the decentralized search process has the minimum expected delivery?

## **Food For Thought**

Try to think of either an application of decentralized search in technological/social networks -- or of an instance of decentralized search in natural networks. For example, back in the early 2000s, when peer-to-peer applications such as Gnutella were popular for sharing music files, decentralized search was at the “*peak of its glory*”.